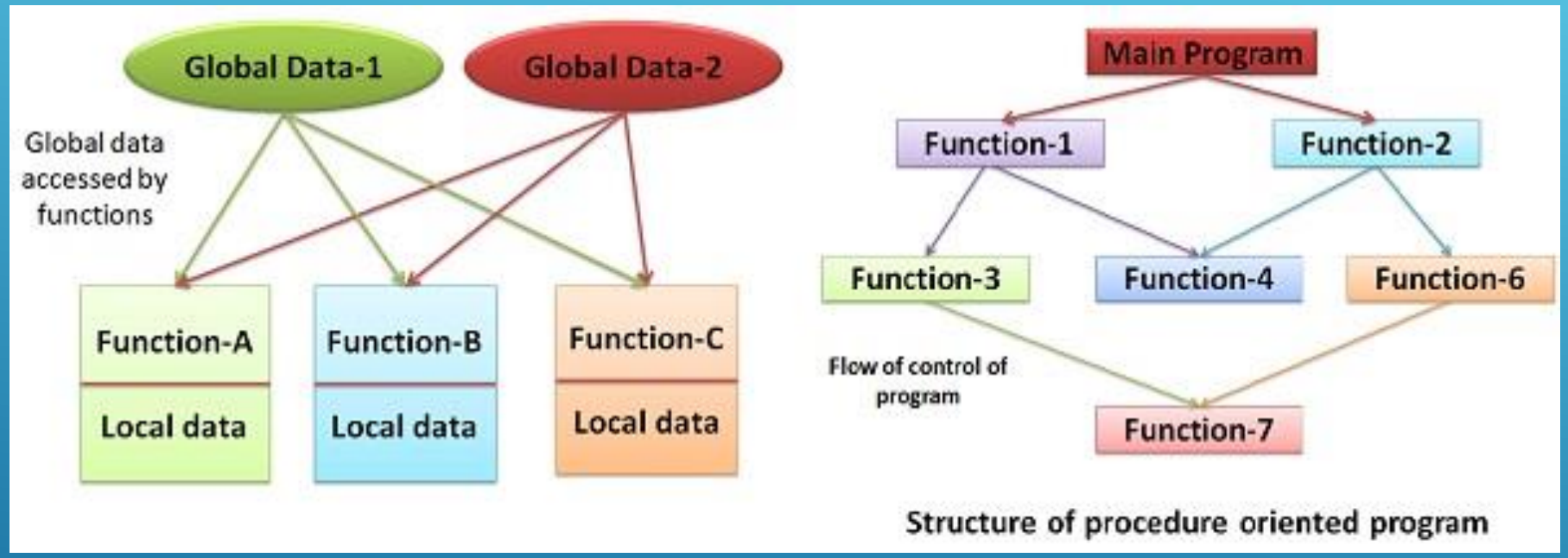
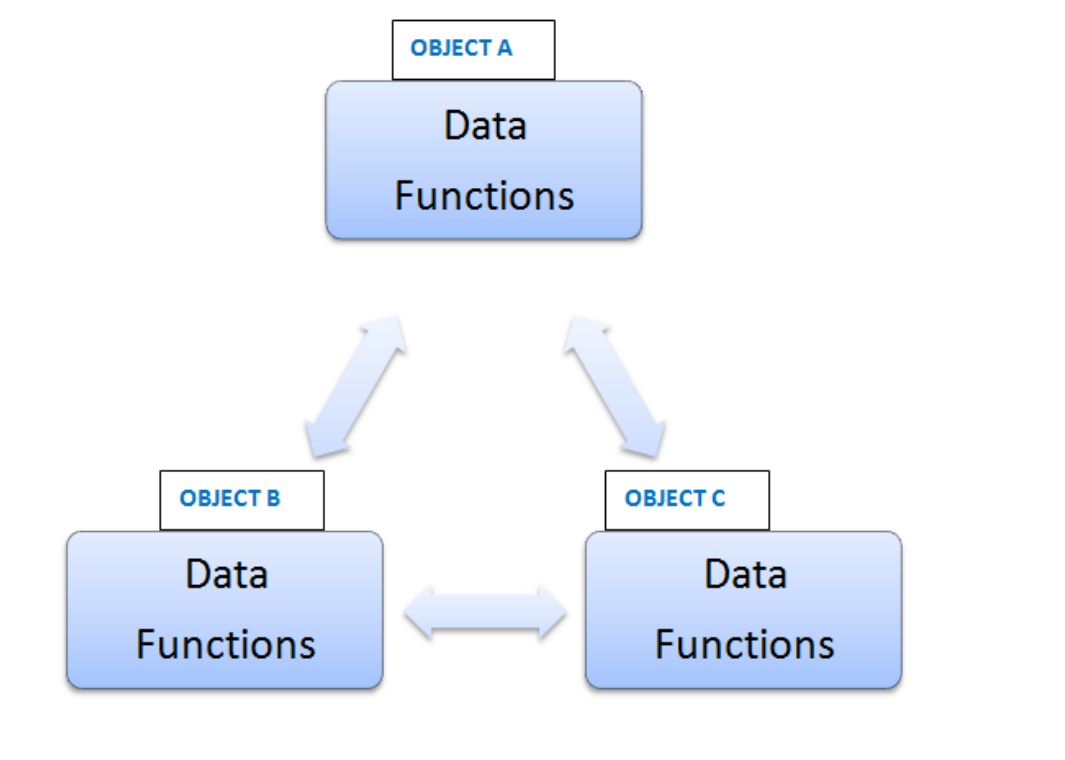
Object programming essentials

POP



http://techdifferences.com/difference-between-oop-and-pop.html

OOP



http://www.circuitstoday.com/difference-between-procedure-oriented-and-object-oriented-programming



OOP VS POP

OOP VS POP

	OOP	POP
Abbreviations	It stands for 'Object Orientated Programming'.	It stands for 'Procedural Orientated Programming'.
Programs	The main program is divided into small object depending on the problem.	The main program is divided into small parts depending on the functions.
Linking	The functions of the objects are linked with other objects by using the message passing.	The different parts of the program are connected with each other by parameter passing and using the operating system.
Data	The data and functions of each individual object act like a single unit.	Every function contains different data.
Importance	Data gets more importance than functions in program.	Functions or algorithms get more importance than data in program.
Data control	Each object controls its own data.	Most of the functions use global data.
Transfer	Data does not possibly transfer from one object to another.	Same data may be transferable from one function to another.
Data hiding	Data hiding is possible, which prevents illegal access of the function from outside of it.	There is no perfect way for data hiding.
Communication	One object links with the other using the message passing.	Functions communicate with other functions maintaining as usual rules.

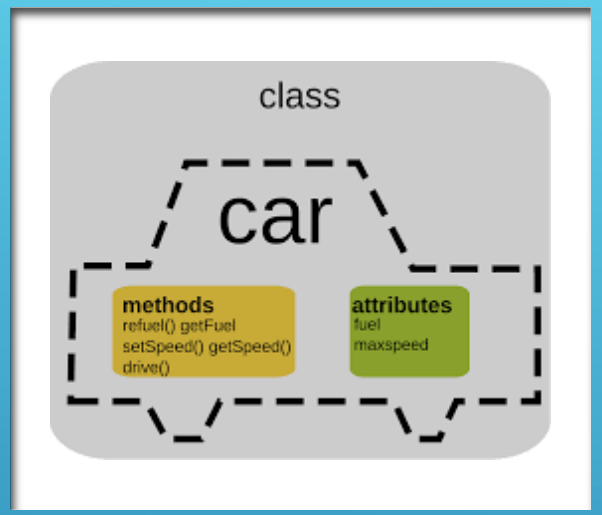
POP VS OOP CONT.

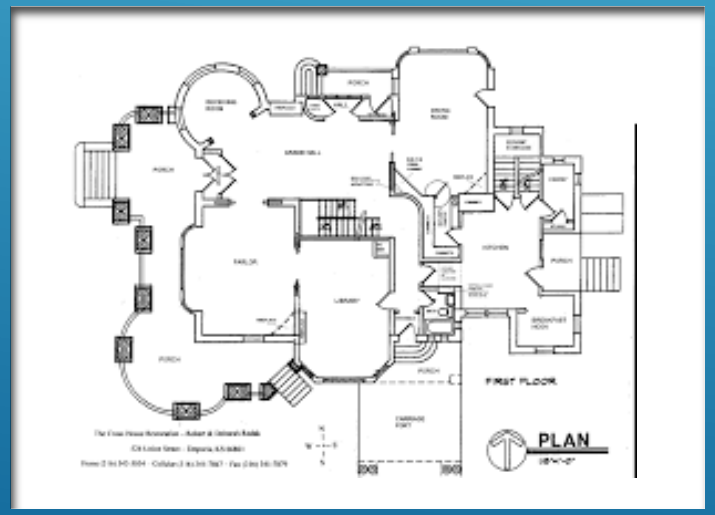
Addition	**More data or functions can be added with the program if necessary. For this purpose, the full program need not be changed.**	**More data or functions cannot be added with the program if necessary. For this purpose, the full program needs to be change.**
Permission	Message passing ensures the permission of accessing member of an object from another object.	To add new data in program, the user should ensure that the function allows it.
Process	Bottom up process is followed for program design.	Top down process is followed for program design.
Overloading	Overloading is possible in the form of Function Overloading and Operator Overloading.	Overloading is not possible.
Access	Public, private, and protected access specifiers are used.	No access specifiers are used.
Examples	C++, Java.	Pascal, Fortran

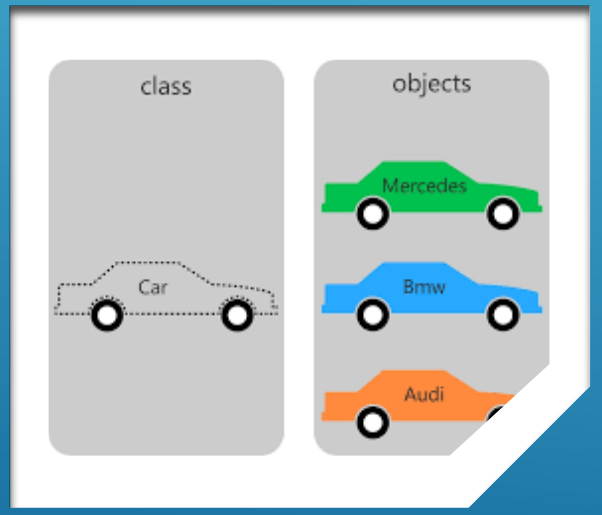
OBJECT ORIENTED PROGRAMMING

- An approach that provides a way of modularizing programs by creating partitioned memory area for both data & functions that can be used as templates for creating copies of such modules on demand.







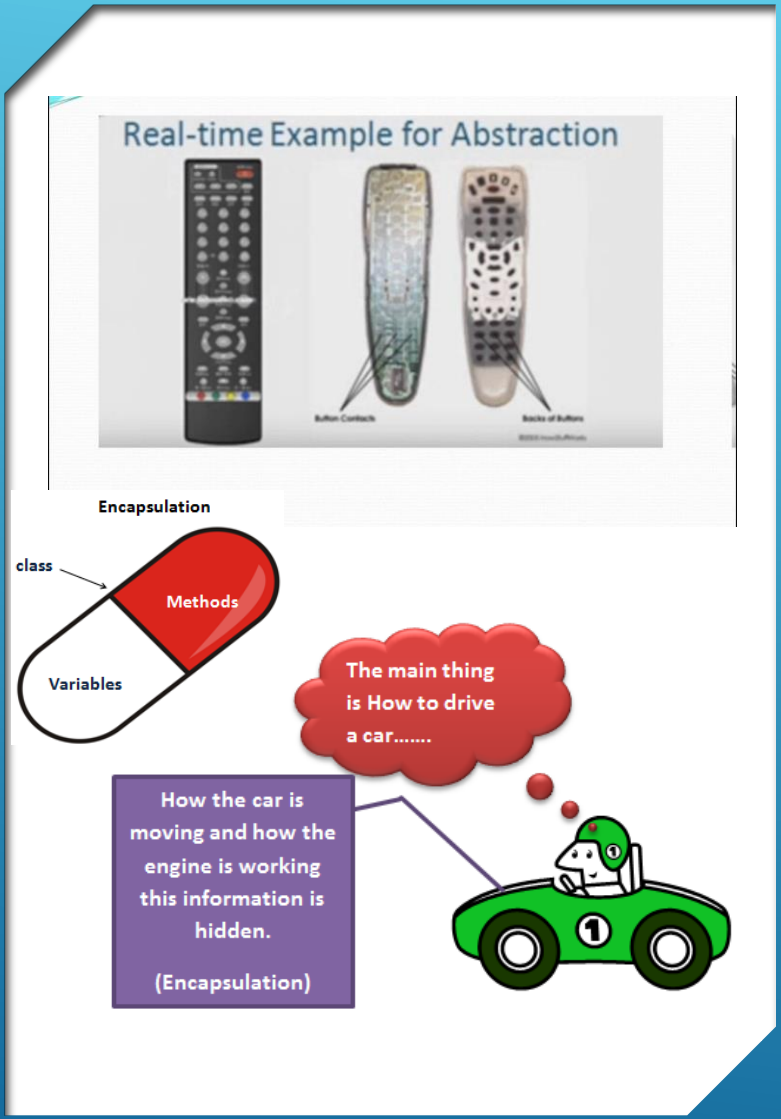


BASIC CONCEPTS/COMPONENTS OF OOP

Class: A class is a way of binding data & associated functions together.

- Classes are user defined data types that act like built in data types.

Object: Objects are instances of a class.

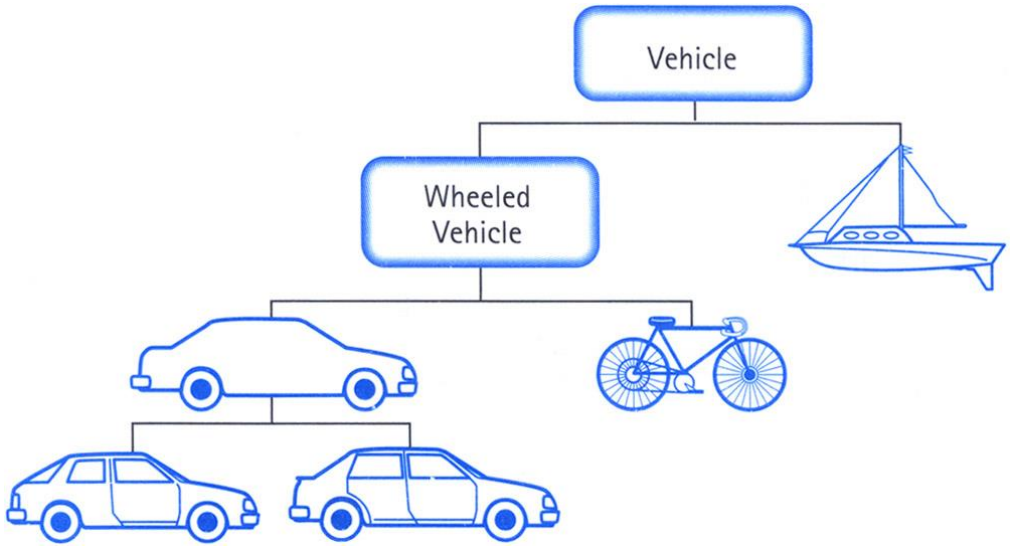


Data abstraction & encapsulation:

- The wrapping of data & functions together is known as encapsulation.
- Prevention of accessing of data of a class for other class objects is known as data abstraction.

BASIC CONCEPTS/COMPONENTS OF OOP

https://datayo.wordpress.com/2015/10/04/object-oriented-programming-a-k-a-oop/



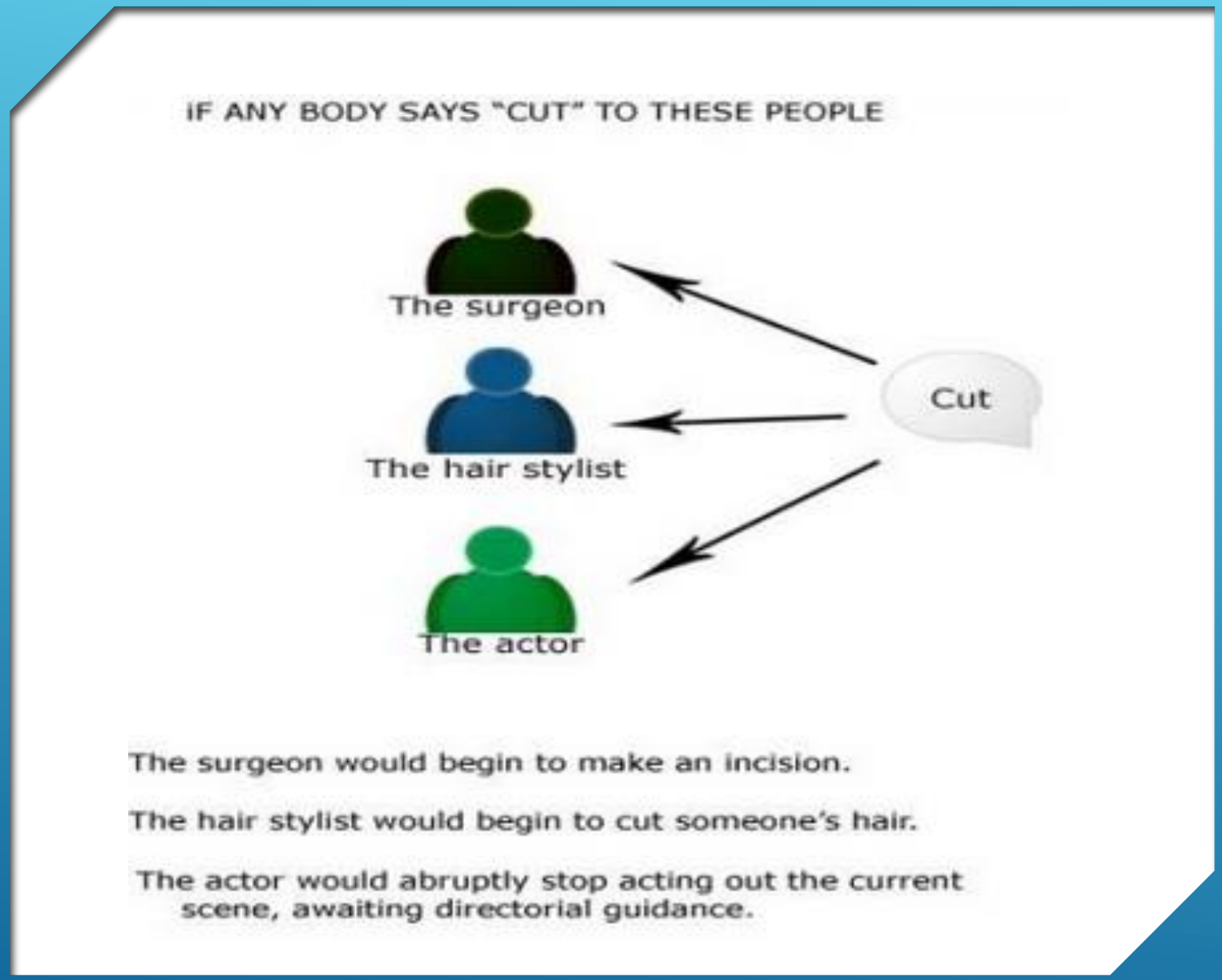


BASIC CONCEPTS/COMPONENTS OF OOP

Inheritance: A mechanism that helps objects of one class to inherit properties from objects of other class.

- Inheritance supports re-usability.

https://nshahpazov.github.io/new-levels-of-abstractions-with-angularjs-and-es6-classes/



BASIC CONCEPTS/COMPONENTS OF OOP

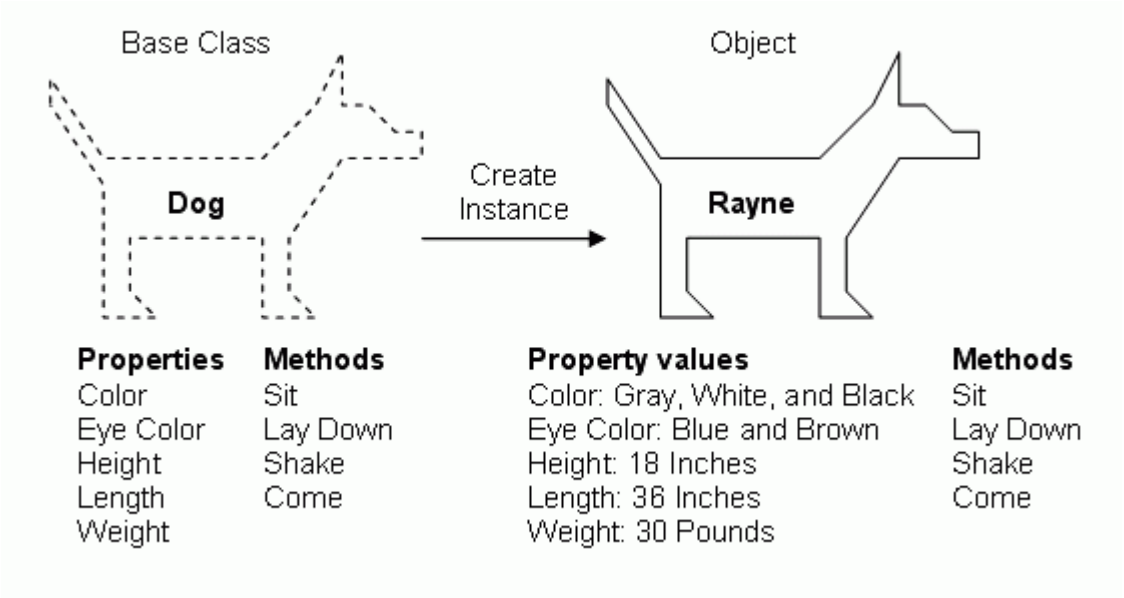
- Polymorphism: A same action can cause different reaction from different objects.

EXAMPLE:

```
class student
{
    int id;
    char name[50];
    };
student s1;
```

- Here s1 is an object of student class.

CLASS AND OBJECT



- Class: A class is a way of binding data & its associated functions together.
- Data of a class are called member data.
- Functions of a class are called member function.

ACCESS MODIFIERS

- Access Modifiers can define accessibility and can be applied to both classes and class members (sub-classes, fields and methods).
- Some of the modifier keywords are shared by both classes and methods.
- When using access modifiers only one can be used at a time.

SCOPE, VISIBILITY, ACCESSIBILITY

- Scope refers to the boundaries of programs and components of programs.
- Visibility refers to portions of the scope that can be known to a component of a program.
- Accessibility refers to portions of the scope that a component can interact with.

SCOPE

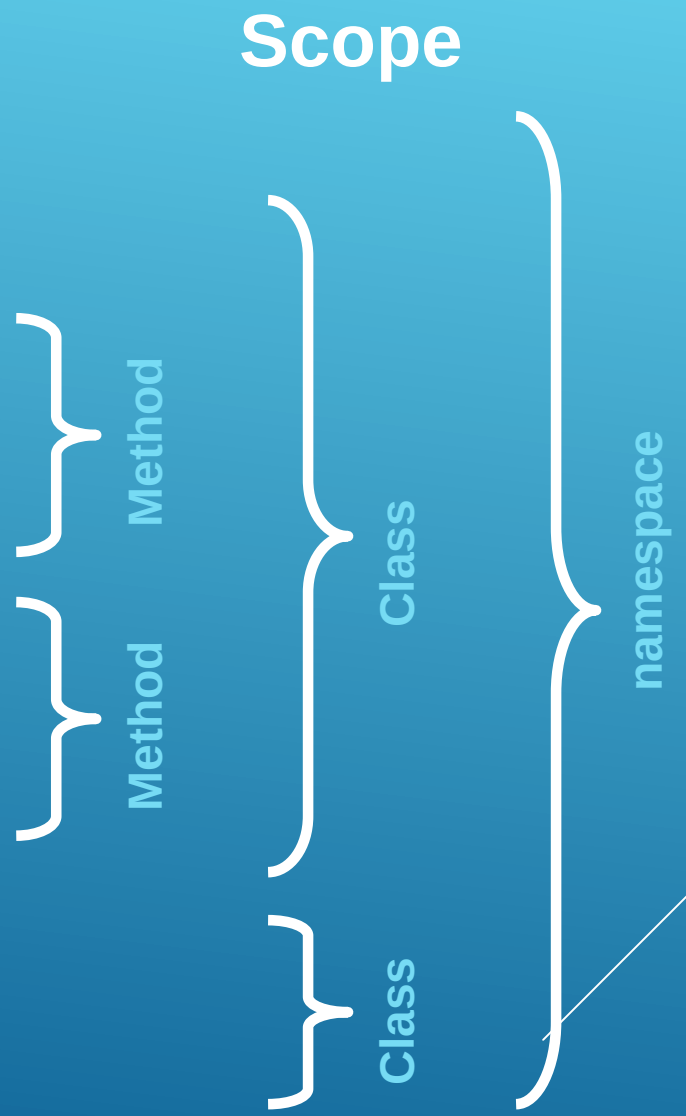
- Scope
  - Part of program where a variable may be referenced
  - Determined by location of variable declaration
    - Boundary usually demarcated by { }
- Example

```
public MyMethod1() {
        int myVar;
        ...
}
```

myVar accessible in method between { }

SCOPE – EXAMPLE

```
class MyClass1 {
void MyMethod1() {
        ...
    }
void MyMethod2() {
        ...
    }
}
    class MyClass2 {
}
```



ACCESS MODIFIER

- A private member is a member which is a accessible only within the class.
- A public member is a member which is

1. Accessible within the class.
2. Accessible from outside the class by using an object of that class & the dot(.) operator.

- By default the member of a class are private.
- Usually the data member are kept as private & member functions are kept as public.
- An object is an instance of a class.

EXAMPLE:

```
class item
{
private:
    int id; // member data
    double price;
    char *name;
public:
    void getdata() //member function
    {
      name=new char[50];
      cin>>id>>price>>name;
    }
    void putdata()
    {
      cout<<"id="<<id<<"price="
      <<price<<"name="<<name<<endl;
    }
};
```

```
int main()
{
        item t1;
        t1.getdata();
        t1.putdata();
        return 0;
}
```

STATIC MEMBER DATA

- Only one copy of the variable for the entire class no matter how many objects are declared.
- By default data members are initialized by zero.
- Visible only within the class.
- Must be defined outside the class definition.

EXAMPLE:

```
class item
{
private:
    int id;
    double price;
    static int count;
public:
    void getdata()
    {
      cin>>id>>price;
      count++;
    }
void putdata()
    {

      cout<<"id="<<id<<"price="<<
    price<<endl;
    }
void showcount()
{
    cout<<"count="<<count<<endl;
}
};
int item::count;

int main()
{
    item t1;
    t1.getdata();
    t1.showcount();
    item t2;
    t2.getdata();
    t2.showcount();
    return 0;
}
```

Count = 1

Count = 2

STATIC MEMBER FUNCTION

- A static member function can be accessed even without creating an object.
- A static member function can deal with static member data only.

EXAMPLE: STATIC MEMBER FUNCTION

```
class item
{
private:
    int id;
    double price;
    static int count;
public:
void getdata()
{
    cin>>id>>price;
    count++;
}
void putdata()
{
    cout<<"id="<<id<<"price="<<
    price<<endl;
}
```

```
static void showcount()
{
        cout<<"count="<<count<<en
dl;
}
};
int item::count=5;
int main()
{
        item t1;
        t1.getdata();
        t1.showcount();
        item::showcount();

        return 0;
}
```

Output:

Count = 6

Count = 6

THIS POINTER

- **`this`** pointer
  - Allows objects to access their own address
  - Not part of the object itself
  - Implicit first argument on non-**`static`** member function call to the object
  - Implicitly reference member data and functions

EXAMPLE

```
class Class {
public:
void setVal(int value) { this -> value = value; }
void setVal(void) { value = -2; }
int getVal(void) { return value; }
private:
int value;
};
```

FUNCTION OVERLOADING

- Function overloading means to have more than one function with the same mane but with different parameters.
- Overloads functions are differentiated by checking

1. number of arguments.
2. Type & sequence of arguments but not by return type of the function.

EXAMPLE

```
Double volume (double a)
{
   return a*a*a;
}
Double volume (double r, double h)
{
   return 3.14*r*r*h;
}
int main()
{
cout <<"volume of cube="<<volume(5)<<endl;
cout <<"volume of cylinder="<<volume(5,10)<<endl;
return 0;
}
```

CONSTRUCTOR

- A constructor is a member function of a class which is called whenever an object is created.
- Name of the constructor function must be same as the class name.
- It has no return type.
- It must be declared as public.
- It can be overloaded.
- It may /may not have any arguments.
- A constructor that does not take any parameter is known as default constructor.

EXAMPLE

```
class A
{
    int a;
public:
    A()
    {
      cout<<"Default"<<endl;
    }
    A(int x)
    {
      a=x;
      cout<<a<<endl;
    }
};
```

```
int main()
{
    A a1(5);
    A a2;
    A a3;
    a3=A(10);
    A a4=A(20);
    A a5=30;
    return 0;
}
```

Output:

```
5
Default
Default
10
20
30
```

CONSTRUCTORS

- If any constructor is not written then the compiler automatically takes a default constructor.
- If *any* non default constructor is written then the default constructor is not added by the compiler. So, in such case, a default constructor *should* be written by the programmer.

COPY CONSTRUCTOR

- A constructor CAN take any data type or class as its parameter but CANNOT take its own type.
- A constructor CAN take its own reference.
- A constructor that takes its own reference as parameter is known as copy constructor.

EXAMPLE

```
class A
{
    int a;
public:
    A()
    { }
    A(int x)
    {
      a=x;
    }
    A(const A &ob)//copy const
    {
      a=ob.a;
      cout<<a<<endl;
    }
};
```

```
int main()
{
    A ob1(10);
    A ob2=ob1;
    A ob3(ob1);
    A ob4;
    ob4=ob1;//copy cons not
    called
    return 0;
}
```

Output:

10

10

DESTRUCTOR

- A destructor is used to destroy the object.
- It will be invoked automatically as soon as the life of the object is finished.
- It neither has any return type nor it takes any parameter.
- It has same name as the class name but begins with a Tilde (~) sign.
- As like constructor, a default destructor is always written if not explicitly mentioned by the programmer.

DESTRUCTOR

- Destructors are called exactly in the opposite order of the constructor.
- Usually destructors are written only when memory is dynamically allocated in constructors.

EXAMPLE

```
int count=0;
class A
{
    int a;
public:
A()
{
    count++;
    cout<<"Const of object
    no:"<<count<<endl;
}
~A()
{
    cout<<“Dest of object
    no:"<<count<<endl;
    count--;
}

};
```

```
int main()
{
    A a1,a2;
    {
    cout<<"block 1\n";
      A a3;
    }
    A a4;
    {
    cout<<"block 2\n";
      A a5;
    }
    A a6;
return 0;
}
```

OUTPUT:

```
Const of object no:1
Const of object no:2
Block 1
Const of object no:3
Dest of object no:3
Const of object no:3
Block 2
Const of object no:4
Dest of object no:4
Const of object no:4
Dest of object no:4
Dest of object no:3
Dest of object no:2
Dest of object no:1
```